(18 1/2)

COL 7[4]18 **Midterm : 2025-26** **Name (Print):** KHUSHI DESHMUKH

110 Minutes **Total Marks: 50** **Entry #** 2025MCS2104

Closed Book Closed Notes No Electronic Devices [no smartwatches/digital watches].
No Clarifications given in Exam – make Assumptions if you think problem is incompletely/under specified.
Please use **blue/black pen** to write your answers [**Pencil is not acceptable**].
Please write your name OR entry number in each page — if that is absent, grading of that page is **NOT** done.
Please refrain from taking a bathroom break — marks of students who take a bathroom break will be modulated by the marks of a special Viva for these students [*essentially all answers have to defended orally*].

1.

(a) (1 mark) Why does it become difficult to maintain near linear performance gains with deeper pipelines?

Deeper Pipelines have multiple instructions running per cycle time. Whenever exceptions happen like Branch Prediction, Cache Misses, Run time Errors, Cycle Stalls happens which lead to stalling of multiple instructions, hence performance reduces drastically.

0

(b) (1 mark) Why is it necessary to issue instructions in order?

1/2

It is necessary to issue instructions in order →

① If load of any input happens before write, the wrong input can be loaded.

② If 2 instructions WRITE simultaneously, then wrong data could be written.

Program Semantics

Bad Conceptual understanding.

To avoid these happenning and to maintain consistent data, issue instructions in order.

(c) (3 marks) What hardware structure can be used to support branch prediction, data prediction and precise exceptions in an out of order processor? Explain what is this structure and how this helps to recover from an exception or misprediction.

2 1/2

Reorder Buffer is a hardware structure used to support Branch Prediction, Data Prediction etc.

It contain commit → / issue →

I	V	Reg

It contains 2 pointers 'commit' to keep track of instructions which have written output. and 'issue' for new instructions.

It makes sure, the output is mapped to Register, so that whenever exception / Branches happens our data doesn't lose.

Reorder Buffer helps in maintain order of Instructions.

flush? etc.

2

(a) (2 marks) Explain with an example, why a larger dependence distance has a larger potential for unrolling.

A large Dependence Distance has larger Potential for Unrolling. because many instructions would be between the dependant instructions. Possible anomalies →
① Wrong data loading can happen (chances are more)
② Simultaneous writes / Reads
Thus more instructions in between would lead to the chances of anomalies increasing, hence larger potential for loop Unrolling.

example

(b) (3 marks) Consider the following loop:

```
for (i = 0; i < 10; i++) {
        A[3*i + 1] = A[2*i + 1] + 5;
}
```

(a) Use the GCD test to determine whether a loop-carried dependence may exist between the array references.

GCD Test → To determine whether loop carried dependence may exist b/w array references.

i=1 A[3+1] = A[3] + 5
i=2 A[7] = A[5] + 5

loop carried dependencies don't occur

) Verify whether a true loop-carried dependence actually exists within the loop bounds.

No Yes, loop carried Dependence does not exists between the loop Bounds.
. In the loop for (i=0; i<10; i++) {---
If the loop of i = 5 runs before i = 4, there would be no problem at all.
as i = 4 A[13] = A[9] + 5
i = 5 A[16] = A[11] + 5
even if i = 5 runs before i = 4, there would be no data mismarage.

Data Dependence may exist

Name: KHUSHI DESHMUKH

3. Consider the following branch trace for two static branches, B_1 and B_2, executed in program order:

$$B_1: T, T, T, T, T, \ldots$$
$$B_2: T, N, T, N, T, N, \ldots$$

where T = Taken, N = Not Taken.

(a) (2 marks) Explain how a **local branch predictor** would perform on B_1 and B_2.

When B_1 = Taken for first time, local Branch Predictor would update than B_1 = Taken Hence it would perform good on B_1 ✓ Accuracy 1/2

When B_2 = Taken for first time, local Branch Predictor would update B_2 = T but on next instruction B_2 = Not taken, hence it would cause cycle stalls. B_2 would not perform well here.

(b) (2 marks) Explain how a **global branch predictor** using the last outcome of B_1 to predict B_2 would behave.

Global Branch Predictor would look at B_1 and B_2 both unlike local BP.

If B_1 = Taken for last outcome, Global Branch Predictor would assume B_2 = Taken, but it would also look at the previous outcomes of B_2. So! Accuracy

(c) (2 marks) Show why a **tournament predictor** that chooses between local and global predictors could achieve higher accuracy on this trace.

Tournament Predictor choses between local and Global Predictor according to how they perform on the given instructions.

If the branches are consistent like in case of B_1, the branches are T, T, T, T ... . So Tournament Predidor would chose local Predictor.

If the branches are inconsistent like in case of B_2.

T, N, T, N ... , it would chose Global BP which would look at both B_1 and B_2. Accuracy

It also looks at History lengths, what was the history of Branch Predictions, so it would have higher accuracy. 1/2

Name: KHUSHI DESHMUKH

4

Consider the following code snippet (C-style pseudocode):

```
for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            if (A[i][j] != 0) {
                sum += A[i][j] * B[j];

            }

        }

}
```

Answer the following questions. Be concise but precise; where numerical answers are required, state any assumptions you make.

(a) (2 marks) Identify all loop-control branches in the snippet and state where they occur.

for (i=0 ; i<N ; i++) { for (J=0 ; J<N ; J++) { - - - -

This is loop control Branch, Branch is taken multiple times during the code execution.

(2)

(b) (1 mark) Identify the data-dependent branch in the code.

if (A[i][J] != 0) { sum += A[i][J] * B[J]; }

It is data dependent branch, because branch would only be taken when A[i][J] is not equal to 0.

(1)

(c) (1 mark) Explain why the data-dependent branch is generally harder to predict than loop-control branches

In loop control Branches, we already get the inference that branch would be taken multiple times and sometime also get the limit value, prediction becomes easy.

But in Data Dependent Branches, Branch would depend on Particular Data, which keeps on changing, Hence difficult to predict changing data.

(1)

(d) (2 marks) Explain which predictor [from what you have implemented in gem5] will perform well for this workload.

GShare Predictor would work best on this Workload because it gives higher performance on branch heavy code.

Sparse Data

1/2

1

(e) (3 marks) Assume A is a sparse matrix and has 70% zeros (i.e., probability of non-zero $p = 0.30$). What is the expected misprediction rate of an Always-Taken predictor on the data-dependent branch?

A = sparse matrix

Always Taken Predictor predicts that Branch would always be taken

So B = T, T, T, T - - -

P(Branch Taken) = 0.3

P(Branch Predicted) = 1

P(Misprediction) = 0.7

Misprediction Rate = 70%

E

3

(f) (1 mark) What possible gem5 statistics you would collect to evaluate predictor performance for this snippet.

Gem 5 statistics to collect for evaluating predictor performance.

1. Branches Mispredicted during execution.
2. Total Branches during execution.

We can also collect IPC (Instructions per cycle), CPI (Cycle per Instructions) etc. to evaluate Predictor Performance

1

5. Consider the loop (indexing starts at 1):

```
for (i = 1; i < N; i++) {
        A[i] = A[i-1] + B[i];
}
```

The Machine model is as follows:

- Load latency = 2 cycles
- Add latency = 1 cycle
- Store latency = 1 cycle
- Issue width: **at most** one memory operation and one ALU op per cycle.

(a) (2 marks) Compute the minimum initiation interval IImin (resource and recurrence

Load A[i-1] 2

load B[i] 2

Add A[i], A[i-1], B[i] 1

Store A[i] 1

10

pendence remains a bottleneck.

favourable to unroll the loop up to the safe Point. This makes us avoid unecessary handling. We can simply start the loop again.

(c) (3 marks) Suppose the loop is modified to:

```
for (i = 1; i < N; i++) {
        A[i] = A[i-1] + B[i];
        C[i] = C[i-1] * 2;
}
```

Now there are two independent recurrences (one ADD, one MUL). How does this affect scheduling on the VLIW? Can software pipelining help overlap these dependences?

VLIW depends upon compiler for Dependency analysis.
Compiler detects independent instructions before hand
o the instruction level Parallelism Processing (ILP) can be done.
mpiler will detect the two independent
urrences before hand. It would make
run the instructions parallely, which
increase the Parallelism hence
e performance.

Design a *matrix multiplication kernel* in the dataflow model. This requires you to:

- Draw the dataflow graph for multiplying two $N \times N$ matrices.
- Explain how token matching ensures correctness when multiple instances of the same multiplication operation are active simultaneously.

Expected Points in Answer: Graph nodes for multiply, add, accumulation; arcs for row/column data tokens. Token matching unit pairs operands based on tags (row, column indices).

DATA FLOW GRAPH for multiplying two NXN matrices.

$$Y(n) = \sum_{i,j,k=0}^{n} A(i,k) * B(k,j)$$

Graph Nodes →

① INPUT NODES → Matrices of size N with indices (i,k) (k,J).

② MULTIPLY NODES → $A(i,k) * B(k,J) = C[i,J]$

how?

③ Addition Nodes → All the multiply nodes will be added with appropriate coefficient for generating final product (1)

TOKEN MATCHING → Token Matching ensures ~~the correct inputs are processed with each other~~ It ensures no wrong entries gets ~~computed~~.

Generic

Token matching unit will check if both the matrices have 'k' in common.

[i , k , J]

This will ensure we get correct output

The column of first ma = row of second matri

Graph on next page.

Where is the Graph?

(b) (3 marks) Produce a feasible software-pipelined plan (prologue, kernel, epilogue) with that II.

Pipeline ⇒ Used to run instructions parallely so to save time and increase performance.
Software Pipeline Plan ⇒ Detecting Pipeline anomalies like Branches etc during the compilation phase not at Hardware Level. making it feasible as Simple Hardware & low computation costs.

(c) (1 mark) Explain whether you can have a 1 iteration per cycle?

(d) (1 mark) Compare steady-state throughput to the naïve loop.

(e) (2 marks) What does support for vector chaining and tailgating allow in a vector processor?

Vector Chaining ⇒ When we have 2 vector operations. Eg → Add C,A,B Multiply E,C,D we have to wait for 1'st operation to finish to start 2nd one.
But in Vector chaining, we simultaneously take first output of Vector addition $C_1 = A_1 + B_1$ and feed it to Multiplier (2nd op) and thus it increases performance.

Tail Gating → When the input is less than the size of Vector, addition bits are added (dummy) so that operation can be performed without reducing size.

Name: KHUSHI DESHMUKH

6. Consider the loop:

```
for (i = 1; i < N; i++) {
        A[i] = A[i-1] + B[i];
}
```

Assume a VLIW processor with the following properties:

- 2 ALUs (ADD or MUL, latency = 1 cycle).
- 2 Load/Store units (load latency = 2 cycles, store = 1 cycle).
- Each VLIW bundle can issue up to 4 instructions (2 ALU + 2 LD/ST).
- Fully pipelined functional units.
- Perfect branch prediction and memory disambiguation.

(a) (3 marks) 1. Schedule one iteration without unrolling.

VLIW = Very large Instruction Word.

i=1 Load A[i], B[i] into Register = 2+1 Store

Add A[i], A[i-1], B[i] = 1

Above was one iteration without unrolling.

1/2

2. Show cycle-by-cycle execution and compute the iteration latency.

Cycle by cycle execution ⇒ As each VLIW bundle can issue up to 4 instructions...

~~1st cycle~~.

	1	2	3	4	5	6	7	8
I_1	Load	Load	Add					
I_2		~~Load~~	Load	Load	Add			

i=1 Load R1, A[0]
Load R2, B[0]
Add A[i], A[0], B[i]

1/2

(b) (3 marks) Analyze whether unrolling by 4 [or more] increases throughput or if the

In loop carried Dependencies, often unpl
Branches, exceptions occur which cau
Stalls. To avoid the mismanage,